

# ESCUELA COLOMBIANA DE INGENIERÍA

## DESARROLLO ORIENTADO POR OBJETOS

Diseño y Pruebas.

2026-2 — Laboratorio 2/6

Responsables: José Luis García, Kevin David Quezada

### PARTE I

A Revisando línea base.

1. El proyecto “miniTunes” contiene una construcción parcial del sistema. Revisen el directorio donde se encuentra el proyecto. Describan el contenido en términos de:

- a. Directorios

R/=

Al descomprimir el archivo mminiTunes.zip desde Ubuntu, se genera la carpeta miniTunes (Primer directorio).

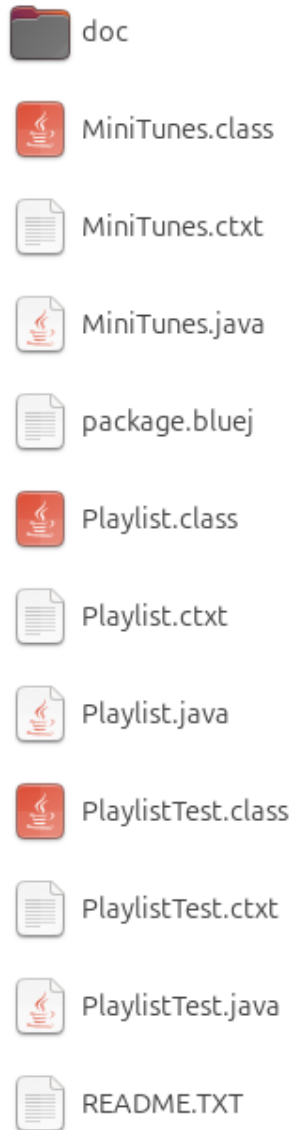


miniTunes

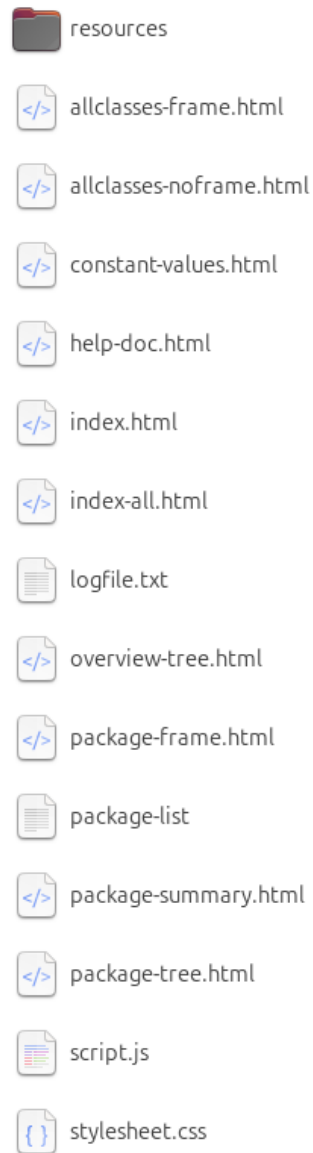


miniTunes.zip

Dentro del primer directorio se encuentra miniTunes (Segundo directorio)



Dentro de miniTunes se encuentra doc (Tercer directorio)



Dentro de doc se encuentra resources (Cuarto y último directorio)

b. Las extensiones de los archivos.

R/= Según las capturas del punto anterior:

.astah

.class

.ctxt

.java

.bluej

.TXT

.gif (dentro de resources)

2. Exploren el proyecto en BlueJ

a. ¿Cuántas clases tiene?

R/=

3 clases que son: playlist, MiniTunes y PlaylistTest.

b. ¿Cuál es la relación entre ellas?

R/=

Tienen relaciones de dependencia (representadas por las flechas punteadas). MiniTunes depende de Playlist para gestionar las listas de reproducción, y PlaylistTest depende de Playlist para ejecutar las pruebas unitarias.

c. ¿Cuál es la clase principal de la aplicación?

R/=

La clase principal es **MiniTunes**.

d. ¿Cómo la reconocen?

R/=

Se reconoce porque es la clase contenedora principal del sistema (la que administra y coordina las listas de canciones en la arquitectura de la aplicación); En Java estándar, la clase principal es aquella que contiene el método de entrada public static void main(String[] args). En el diagrama, MiniTunes es la clase "origen" o "controladora" de la que dependen las demás (como Playlist).

e. ¿Cuál es la clase “diferente”?

R/=

La clase diferente es **PlaylistTest**

f. ¿Cuál es su propósito?

R/=

Su propósito es albergar y ejecutar las pruebas unitarias (usando JUnit) para verificar el correcto funcionamiento de los métodos de la clase Playlist bajo la metodología BDD.

3. Generen y revisen la documentación del proyecto: ¿está completa la documentación de cada clase? (Detallen el estado de documentación: encabezado y métodos)

**No está completa.** Como se puede ver en la tabla de clases:

- La clase MiniTunes muestra una pequeña descripción ("MiniTunes.java"), lo que indica que tiene al menos un encabezado básico de Javadoc.

- Las clases Playlist y PlaylistTest aparecen completamente **vacías** en la columna de descripción. Esto significa que no tienen comentarios Javadoc configurados en el encabezado de la clase ni en sus métodos, por lo que la documentación está incompleta o ausente para ellas.
4. Revisen el código fuente del proyecto, ¿en qué estado está cada clase? (Detallen el estado de las fuentes considerando dos dimensiones:
- a. la primera, atributos y métodos
    - **MiniTunes:** Tiene definido un atributo (playlists de tipo TreeMap) y la firma de todos sus métodos principales (define, assign, size, toString, assignUnary, assignBinary, ok), pero **sus cuerpos están vacíos**.
    - **Playlist:** Tiene las firmas de sus constructores y métodos principales (add, delete, select, size, toString, equals), pero **no tiene declarados atributos explícitos** y sus métodos están sin implementar
    - **PlaylistTest:** Está completa en cuanto a su estructura de pruebas unitarias (métodos con @Test, @Before, @After y uso de assertEquals), lista para evaluar el comportamiento una vez se implemente la clase Playlist.
  - b. código, documentación y comentarios)
    - **MiniTunes:** (@author ESCUELA 2026-02) y algunos comentarios en línea de una sola línea (//) que explican brevemente para qué sirve cada método, pero el código ejecutable de los métodos está ausente.
    - **Playlist:** No tiene documentación Javadoc en su encabezado de clase ni en sus métodos. Sin embargo, tiene un bloque extenso de comentarios (//) al inicio que detalla las reglas del negocio (requisitos de las canciones, títulos únicos, duraciones y calificaciones válidas) y un ejemplo en texto de cómo debe verse el método toString().
    - **PlaylistTest:** Tiene comentarios de documentación Javadoc descriptivos en los métodos de configuración (setUp y tearDown), y el código de las pruebas está claramente escrito.

- c. ¿Qué diferencia hay entre el código, la documentación y los comentarios?
- **El código:** Es el conjunto de instrucciones formales escritas en Java (sintaxis ejecutable, declaraciones de clases, métodos, tipos de datos y algoritmos) que la máquina compila y ejecuta.
  - **La documentación (Javadoc):** Son bloques especiales de texto estructurados con etiquetas formales (como `@author`, `@param`, `@return`) ubicados antes de clases o métodos. Su propósito principal es generar documentación oficial externa (como las páginas web HTML de ayuda del API) para que otros desarrolladores entiendan cómo usar las clases.
  - **Los comentarios:** Son notas de texto libre (generalmente con `//` o `/* ... */`) escritas por los programadores directamente dentro del código fuente. Sirven para explicar detalles internos, dejar notas aclaratorias, describir reglas de negocio (como las restricciones de las canciones en Playlist) o documentar temporalmente fragmentos de código, pero **no** forman parte de la documentación oficial generada automáticamente.

B Realizando ingeniería reversa.

## MDD: MODEL DRIVEN DEVELOPMENT

1. Completen el diagrama de clases correspondiente al proyecto. (No incluyan la clase de pruebas)
2.
  - a. ¿Cuáles contenedores están definidos?

R/=

- `List<Song>` (implementado como `ArrayList`) en `Playlist`, para las canciones.
- `TreeMap<String, Playlist>` en `MiniTunes`, para las listas de reproducción indexadas por nombre.

El "contenedor nuevo" al que se refiere el punto 2 es el `TreeMap`.

- b. ¿Qué diferencias hay entre el nuevo contenedor y el ArrayList y el vector [] que conocemos? Consulte el API de java.

R/=

- TreeMap es un **Map** (pares clave-valor); ArrayList y el arreglo [] son colecciones **secuenciales indexadas por posición**.
- TreeMap **ordena automáticamente** sus claves (orden natural o por Comparator); ArrayList/[] conservan simplemente el orden de inserción.
- El acceso en TreeMap es **por clave** ( $O(\log n)$ , vía árbol rojo-negro balanceado); en ArrayList/[] es **por índice** ( $O(1)$ ).
- TreeMap **no permite claves duplicadas**; ArrayList sí permite elementos repetidos, y el arreglo [] tiene **tamaño fijo**, mientras que ArrayList y TreeMap son dinámicos.

3. En el nuevo contenedor

- a. ¿Cómo adicionamos un elemento?

R/=

playlists.put(name, playlist) — si la clave ya existe, reemplaza el valor.

- b. ¿Cómo lo consultamos?

R/=

playlists.get(name) (un valor), o playlists.keySet() / entrySet() para

- c. ¿Cómo lo eliminamos?

R/=

playlists.remove(name).

## PARTE II Aprendiendo a probar

### A Conociendo Pruebas en BlueJ

#### BDD

1. Revisen el código de la clase PlaylistTest

- a. ¿cuáles etiquetas tiene (componentes con símbolo @)?

@Before, @Test, @After

- b. ¿cuántos métodos tiene?

Tiene un total de **6 métodos** (setUp, shouldCreateAEmptyPlaylist, shouldCreateAPlaylist, shouldNotCreateABadPlaylist, shouldRecognizeEqualPlaylists y tearDown).

c. ¿cuántos métodos son de prueba?

Son **4 métodos** de prueba

d. ¿cómo los reconocen?

Se reconocen porque cada uno de ellos está precedido por la anotación o etiqueta @Test

2. Ejecuten los tests de la clase PlaylistTest. (click derecho sobre la clase, Test All)

a. ¿cuántas pruebas se ejecutan?

Se ejecutan **4 pruebas** (correspondientes a los 4 métodos etiquetados con @Test).

b. ¿cuántas pasan?

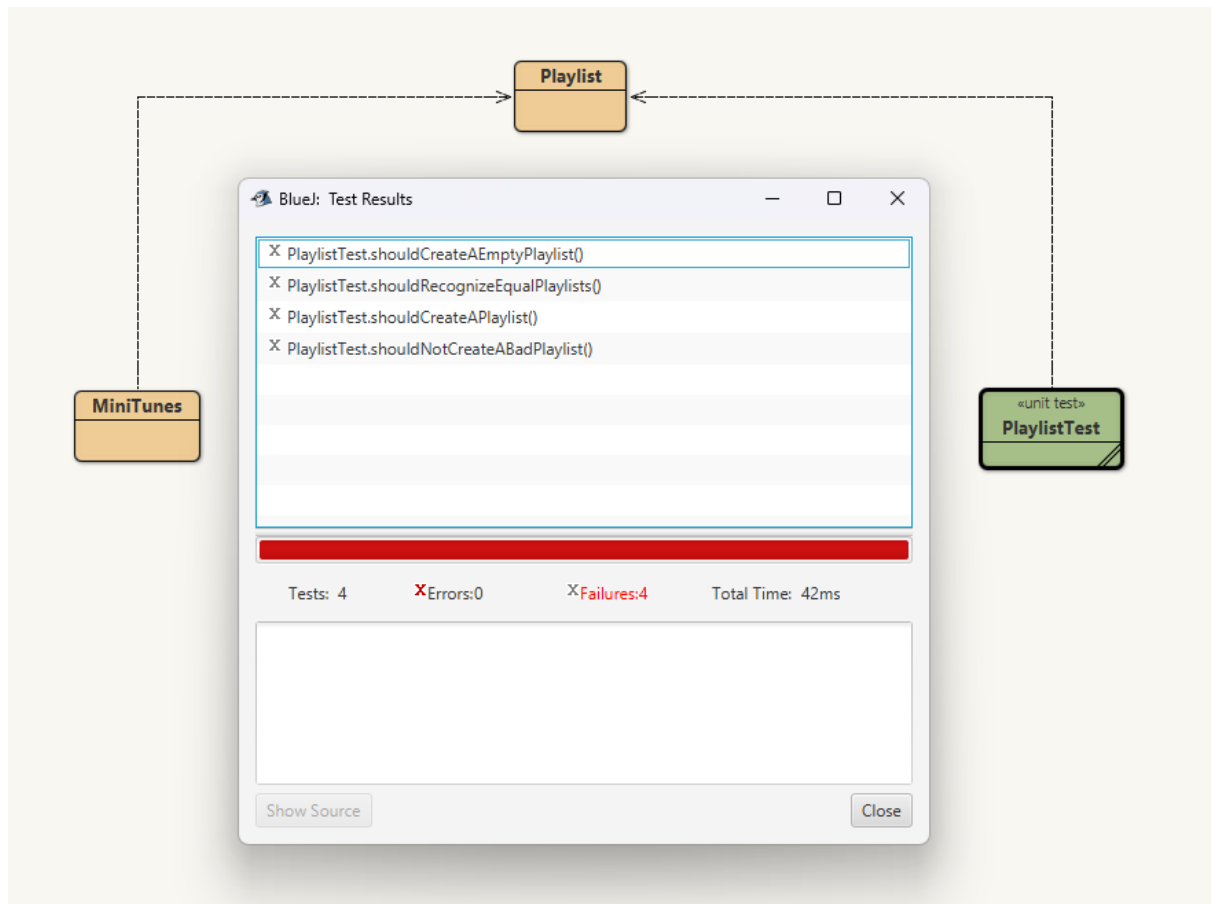
Pasan **0 pruebas**. Todas fallan o dan error.

c. ¿por qué?

Porque los métodos de la clase Playlist aún no tienen la lógica programada; retornan valores por defecto (como -1, null o false), por lo que las aserciones (assertEquals) no coinciden con lo que las pruebas esperan.

d. Capturen la pantalla.





3. Estudien las etiquetas encontradas en 1. Expliquen en sus palabras su significado.
- **@Before**: Indica que el método decorado se ejecutará automáticamente **antes de cada** método de prueba, ideal para preparar el entorno o inicializar objetos de prueba.
  - **@Test**: Señala que el método en cuestión es un **caso de prueba unitario** que el framework JUnit debe ejecutar y evaluar.
  - **@After**: Indica que el método se ejecutará automáticamente **después de cada** prueba, utilizado comúnmente para limpiar recursos o liberar memoria.
4. Estudie los métodos `assertTrue`, `assertFalse`, `assertEquals`, `assertNull` y `fail` de la clase `Assert` del API JUnit 1. Explique en sus palabras que hace cada uno de ellos
- **`assertTrue(condition)`**: Evalúa que una condición lógica sea verdadera (`true`); si resulta ser falsa, la prueba falla.

- **assertFalse(condition)**: Evalúa que una condición lógica sea falsa (false); si resulta ser verdadera, la prueba falla.
- **assertEquals(expected, actual)**: Compara dos valores para verificar si son estrictamente iguales; si no coinciden, la prueba falla.
- **assertNull(object)**: Comprueba que un objeto pasado como parámetro sea nulo (null); si contiene alguna referencia, la prueba falla.
- **fail()**: Fuerza de manera incondicional el fallo inmediato de la prueba en el punto donde se invoque.

5. Investiguen y expliquen la diferencia que entre un fallo y un error en Junit. Escriba código, usen-comporten como lo prometen shouldPass, shouldFail, shouldErr.

- Un **fallo (Failure)** ocurre cuando una aserción explícita no se cumple (por ejemplo, cuando assertEquals compara dos valores y no son iguales).
- Un **error (Error)** ocurre debido a una excepción no controlada e imprevista durante la ejecución del código de la prueba (como un NullPointerException o IndexOutOfBoundsException) que interrumpe la ejecución de forma anormal.

```
@Test
public void shouldPass() {
    // Pasa porque la aserción es verdadera
    assertTrue(true);
}
```

```
@Test
public void shouldFail() {
    // Falla (Assertion Failure) porque los valores no coinciden
    assertEquals(5, 2);
}
```

```
@Test
public void shouldErr() {
```

```
// Da error porque genera una excepción no controlada
(ArithmeticException: división por cero)
int x = 10 / 0;
}
```

## B Practicando Pruebas en BlueJ

### BDD

1.

- a. Determinen los atributos de la clase Playlist.

Una estructura para almacenar las canciones, por ejemplo, un `ArrayList<Song>` o una lista interna (o incluso una estructura basada en arreglos/matrices si la especificación lo pide estrictamente así, pero un `ArrayList` o `HashSet` gestionado mediante una clase interna o estructura de arreglos de strings `String[][]` procesados facilita el almacenamiento). Basado en los tests, se maneja información tabular de canciones (`String[][]`). Podemos usar un contenedor de canciones o una lista de arreglos.

- b. Justifiquen la selección.

Se necesita una estructura dinámica (como `ArrayList`) porque una lista de reproducción puede variar de tamaño: se le pueden agregar o eliminar canciones libremente conforme se ejecuten operaciones.

2.

- a. Determinen el invariante de la clase Playlist.

- No pueden existir dos canciones con la misma combinación exacta de **título y artista** (la unicidad debe respetarse).
- La duración de cada canción (cuando está presente) debe ser un valor entero entre 1 y 9.
- La calificación (rating) debe ser nula o estar compuesta exclusivamente por asteriscos (\*) con una longitud entre 1 y 5.
- El título y el artista son obligatorios (no pueden ser nulos ni vacíos).

- b. Justifiquen la decisión.

Estas restricciones garantizan la consistencia de los datos del dominio según las reglas de negocio especificadas en los comentarios del

código fuente de Playlist y evitan que se ingresen canciones mal formadas (como se evalúa en el test `shouldNotCreateABadPlaylist`).

3. Implementen los métodos de Playlist necesarios para pasar todas las pruebas definidas.
  - a. ¿Cuáles métodos implementaron?
    - **El constructor `Playlist(String [][] songs)`:** Para procesar la matriz de entrada, limpiar espacios, convertir a mayúsculas según las reglas de formato, validar el invariante (descartando canciones inválidas o repetidas) y poblar la lista interna.
    - **`int size()`:** Para retornar el número actual de canciones válidas almacenadas en la lista.
    - **`boolean equals(Playlist pl)` y `boolean equals(Object o)`:** Para comparar dos listas de reproducción y determinar si contienen exactamente las mismas canciones (ignorando mayúsculas, minúsculas y espacios innecesarios, tal como se evalúa en `shouldRecognizeEqualPlaylists`).
    - *(Opcionales para completar la clase, aunque requeridos en ciclos posteriores):* `add`, `delete`, `select`, `toString`.
4. Capturen los resultados de las pruebas de unidad.

### PARTE III. Desarrollando el proyecto miniTunes.

#### MDD-BDD

Para desarrollar esta aplicación vamos a considerar algunos ciclos. En cada ciclo deben realizar los pasos definidos a continuación.

I Definir los métodos base de correspondientes al mini-ciclo actual.

II Definir y programar los casos de prueba de esos métodos. Piensen en los debería y los no Debería (`should...` y `shouldNot...`)

III Diseñar los métodos Usen diagramas de secuencia. En astah, creen el diagrama sobre el método correspondiente.

IV Escribir el código correspondiente (no olvide la documentación)

V Ejecutar las pruebas de unidad (vuelva a 3 (a veces a 2), si no están en verde)

VI Completar la tabla de clases y métodos. (Al final del documento)

## Ciclo 1 — Operaciones básicas

*Definir el nombre de una lista, asignar a un nombre una lista, consultar nombres de listas y consultar las canciones de una lista.*

### I. Métodos base

- ☐ `define(String name)` — define un nuevo nombre de lista de reproducción, inicialmente vacía.
- ☐ `assign(String a, String[][] playlist)` — asigna una lista construida a partir de datos crudos a un nombre ya definido.
- ☐ `size(String a)` — consulta el número de canciones de una lista.
- ☐ `toString()` — consulta los nombres de las listas definidas, en orden alfabético.
- ☐ `toString(String name)` — consulta la representación textual de una lista.

### II. Casos de prueba (should... / shouldNot...)

- ☐ `shouldDefineANewPlaylistName / shouldNotRedefineAnExistingPlaylistName`
- ☐ `shouldAssignAPlaylistToADefinedName / shouldNotAssignToAnUndefinedName`
- ☐ `shouldReturnZeroSizeForANewlyDefinedPlaylist / shouldReturnMinusOneSizeForAnUndefinedName`
- ☐ `shouldListPlaylistNamesInAlphabeticalOrder / shouldReturnEmptyStringWhenNoPlaylistsAreDefined`
- ☐ `shouldReturnStringRepresentationOfAPlaylist / shouldReturnNullStringForAnUndefinedPlaylistName`

### III. Diseño — diagrama de secuencia

*Pendiente*

## IV. Código y documentación

Implementado en las clases MiniTunes y Playlist, con documentación Javadoc en cada clase, atributo y método.

## V. Ejecución de pruebas unitarias

### Ciclo 2 — Operaciones unarias

*Adicionar una canción, eliminar una canción y seleccionar canciones dada una condición.*

#### I. Métodos base

- ☐ assignUnary(String a, String b, char op, String[] values) — asigna a 'a' el resultado de aplicar sobre 'b' la operación unaria 'op' ('a' agregar, 'd' eliminar, 's' seleccionar); delega en Playlist.add / Playlist.delete / Playlist.select.

#### II. Casos de prueba (should... / shouldNot...)

- ☐ shouldAddASongThroughUnaryOperation
- ☐ shouldDeleteASongThroughUnaryOperation
- ☐ shouldSelectSongsThroughUnaryOperation
- ☐ shouldFailWhenSourcePlaylistIsUndefined
- ☐ shouldFailWhenOperatorCharIsInvalid

#### III. Diseño — diagrama de secuencia

*Pendiente:*

## IV. Código y documentación

Implementado en las clases MiniTunes y Playlist, con documentación Javadoc en cada clase, atributo y método.

## V. Ejecución de pruebas unitarias

### Ciclo 3 — Operaciones binarias

*Unión, intersección y diferencia.*

#### I. Métodos base

- ☐ assignBinary(String a, String b, char op, String c) — asigna a 'a' el resultado de combinar 'b' y 'c' con la operación binaria 'op' ('u' unión, 'i'

intersección, 'd' diferencia); delega en Playlist.union / Playlist.intersect / Playlist.difference.

## II. Casos de prueba (should... / shouldNot...)

- ☐ shouldUnionThroughBinaryOperation
- ☐ shouldIntersectThroughBinaryOperation
- ☐ shouldComputeDifferenceThroughBinaryOperation
- ☐ shouldFailBinaryOperationWhenAnOperandIsUndefined
- ☐ shouldFailBinaryOperationWhenOperatorCharIsInvalid

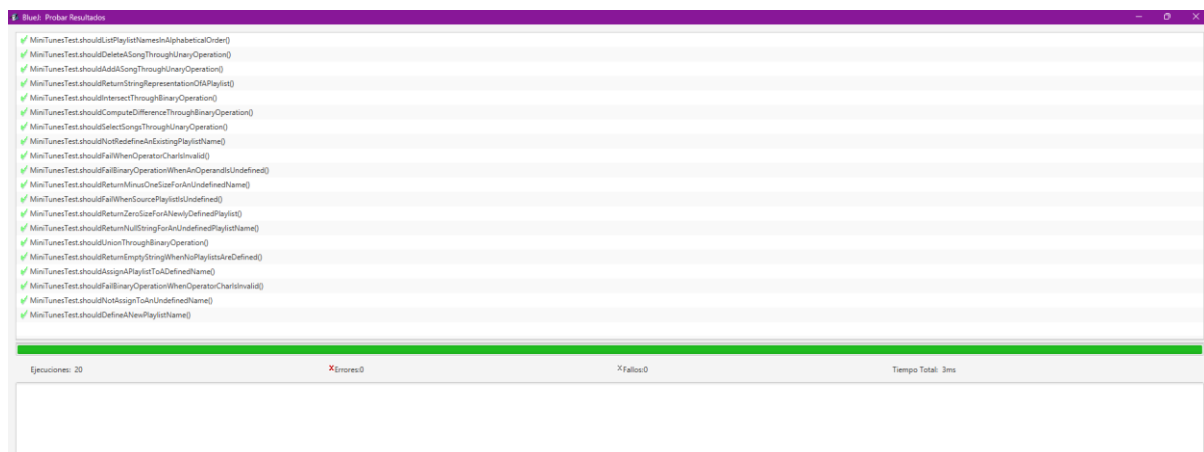
## III. Diseño — diagrama de secuencia

*Pendiente:*

## IV. Código y documentación

Implementado en las clases MiniTunes y Playlist, con documentación Javadoc en cada clase, atributo y método

## V. Ejecución de pruebas unitarias



|                         |                                                                |                   |
|-------------------------|----------------------------------------------------------------|-------------------|
| Blue: Probar Resultados |                                                                |                   |
| ✓                       | PlaylistTest.shouldIntersectTwoPlaylists()                     |                   |
| ✓                       | PlaylistTest.shouldSelectSongsMatchingAGenre()                 |                   |
| ✓                       | PlaylistTest.shouldReturnDifferenceBetweenTwoPlaylists()       |                   |
| ✓                       | PlaylistTest.shouldCreateEmptyPlaylist()                       |                   |
| ✓                       | PlaylistTest.shouldRecognizeEqualPlaylists()                   |                   |
| ✓                       | PlaylistTest.shouldCreateAPlaylist()                           |                   |
| ✓                       | PlaylistTest.shouldAddANewSong()                               |                   |
| ✓                       | PlaylistTest.shouldNotChangeWhenDeletingANonExistentSong()     |                   |
| ✓                       | PlaylistTest.shouldNotAddADuplicateSong()                      |                   |
| ✓                       | PlaylistTest.shouldNotModifyOperandsWhenComputingUnion()       |                   |
| ✓                       | PlaylistTest.shouldSelectAllSongsWhenCriteriaAreAllWildcards() |                   |
| ✓                       | PlaylistTest.shouldNotAddAnInvalidSong()                       |                   |
| ✓                       | PlaylistTest.shouldUnionTwoPlaylistsAvoidingDuplicates()       |                   |
| ✓                       | PlaylistTest.shouldNotCreateABadPlaylist()                     |                   |
| ✓                       | PlaylistTest.shouldDeleteAnExistingSong()                      |                   |
|                         |                                                                |                   |
|                         |                                                                |                   |
| Ejecuciones: 15         | Errores:0                                                      | XFallos:0         |
|                         |                                                                | Tiempo Total: 3ms |

| C<br>i<br>c<br>l<br>o | MiniTunes                                                                          | MiniTunesTest                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1                     | define,<br>assign,<br><br>size,<br><br>toString(<br>),<br><br>toString(<br>String) | shouldDefineANewPlaylistName<br><br>shouldNotRedefineAnExistingPlaylistName,<br><br>shouldAssignAPlaylistToADefinedName,<br><br>shouldNotAssignToAnUndefinedName,<br><br>shouldReturnZeroSizeForANewlyDefinedPlaylist,<br><br>shouldReturnMinusOneSizeForAnUndefinedName,<br><br>shouldListPlaylistNamesInAlphabeticalOrder,<br><br>shouldReturnEmptyStringWhenNoPlaylistsAreDefined, |



|   |              |                                                                                                                                                                                                                                                                   |
|---|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   |              | <p>shouldReturnStringRepresentationOf APlaylist,</p> <p>shouldReturnNullStringForAnUndefinedPlaylistName</p>                                                                                                                                                      |
| 2 | assignUnary  | <p>shouldAddASongThroughUnaryOperation,</p> <p>shouldDeleteASongThroughUnaryOperation,</p> <p>shouldSelectSongsThroughUnaryOperation,</p> <p>shouldFailWhenSourcePlaylistIsUndefined,</p> <p>shouldFailWhenOperatorCharIsInvalid</p>                              |
| 3 | assignBinary | <p>shouldUnionThroughBinaryOperation,</p> <p>shouldIntersectThroughBinaryOperation,</p> <p>shouldComputeDifferenceThroughBinaryOperation,</p> <p>shouldFailBinaryOperationWhenAnOperandIsUndefined,</p> <p>shouldFailBinaryOperationWhenOperatorCharIsInvalid</p> |

## RETROSPECTIVA

1. **Tiempo invertido:** José García — 14 horas. Kevin Quezada — 14 horas.
2. **Estado actual del laboratorio:** Avanzado/casi terminado. Los Ciclos 1, 2 y 3 están implementados, probados (JUnit en verde) y documentados; sólo falta el diagrama de secuencia en astah, las capturas de ejecución y, opcionalmente, el Ciclo 4 (bono).
3. **Práctica XP más útil:** Testing (TDD/BDD con JUnit). Escribir primero los should.../shouldNot... obligó a definir claramente el comportamiento esperado

de Playlist y MiniTunes antes de programar, reduciendo errores de diseño sobre la marcha.

4. **Mayor logro:** Lograr que MiniTunes delegara correctamente en Playlist para las operaciones unarias y binarias, manteniendo ambas clases inmutables y con todas las pruebas en verde.
5. **Mayor problema técnico:** Diseñar la igualdad de canciones en dos niveles (identidad por título+artista vs. igualdad total de atributos) sin que ambos criterios entraran en conflicto en los Set/comparaciones internas. Se resolvió separando `sameEntryAs()` de `equals()`.
6. **Qué hicieron bien / compromiso de mejora:** Se dividieron el trabajo por clases (Playlist/MiniTunes) y se integraron con pruebas ya escritas, lo que evitó reprocesos. Para mejorar, se comprometen a documentar el código a medida que se escribe, en lugar de dejarlo para el final.
7. **Referencias usadas:** Documentación oficial de Oracle para `java.util.TreeMap` y `java.util.List` (Java SE 17 API Specification, [docs.oracle.com](https://docs.oracle.com)), y la guía de JUnit 4 ([junit.org](https://junit.org)) para los métodos de `Assert`.